Practical 5: Implementation of scheduling algo: First come first served [FCFS]

Author: Gaurav Rao

Roll No: 24293916091

Class: CSE A (IInd year)

Date: Januray 23, 2026

Topic: Implementation of scheduling algo: First come first served [FCFS]

1. Theory: First Come First Served (FCFS)

First Come First Served (FCFS) is the simplest type of CPU scheduling algorithm. In this scheme, the process that requests the CPU first is allocated the CPU first. It is implemented using a **FIFO (First-In, First-Out)** queue.

- **Non-Preemptive**: Once the CPU is allocated to a process, that process keeps the CPU until it releases it, either by terminating or by switching to the waiting state (e.g., for I/O).
- **Criteria**: The scheduling is based solely on **Arrival Time**.
- **Pros**: Simple to understand and implement.
- **Cons**:
  - **Convoy Effect**: Short processes behind a long process have to wait, leading to lower CPU utilization.
  - High average waiting time.

Formulas

- **Waiting Time** = Start Time - Arrival Time
- **Turnaround Time** = Completion Time - Arrival Time
  - *(Or: Turnaround Time = Burst Time + Waiting Time)*

.

2. Timeline Visualization

Scenario Data

Process	Arrival Time	Burst Time
P1	0	15
P2	2	3
P3	5	5

Execution Flow (Gantt Chart)

Time Range	Process	Description of Events
0 - 2	**P1**	P1 starts at 0. Executes until P2 arrives.
2	*System*	**{P2 Process Arrives}** assigned to Queue.
2 - 5	**P1**	P1 continues executing.
5	*System*	**{P3 Process Arrives}** assigned to Queue.

Time Range	Process	Description of Events
5 - 15	**P1**	P1 continues executing until completion.
15	**P1**	**[P1 Finishes]**. CPU switches to P2.
15 - 18	**P2**	P2 executes (Waited from 2 to 15).
18	**P2**	**[P2 Finishes]**. CPU switches to P3.
18 - 23	**P3**	P3 executes (Waited from 5 to 18).
23	**P3**	**[P3 Finishes]**.

.

3. C Implementation

This program allows you to input processes and choose between the standard FCFS sequence (based on arrival/input order) or a custom execution sequence.

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_PROCESS 10

struct Process {
  int pid;
  int arrival_time;
  int burst_time;
  int waiting_time;
  int turnaround_time;
  int completion_time;
};

void calculate_times(struct Process output_sequence[], int n) {
  int current_time = 0;
  float total_wt = 0, total_tat = 0;

  printf("\n\n--- GANTT CHART ---\n");
  printf("Time: %d", current_time);

  // Note: In strict FCFS, we usually sort by arrival.
  // Here we process in the order provided by the 'output_sequence' array
  // to support the 'Custom Sequence' feature.

  for (int i = 0; i < n; i++) {
    // If CPU is idle before this process arrives (only applicable if
strictly
    // following arrival times, but for custom sequence we assume user
wants to
    // force run this next)
    if (current_time < output_sequence[i].arrival_time) {
      // For strict FCFS logic, we would jump to arrival time.
      // For this custom sequence menu, we stick to necessary logic:
starts when
      // previous finished OR when it arrives, whichever is later.
```

```
      current_time = output_sequence[i].arrival_time;
    }

    output_sequence[i].waiting_time =
        current_time - output_sequence[i].arrival_time;

    // Waiting time cannot be negative
    if (output_sequence[i].waiting_time < 0)
      output_sequence[i].waiting_time = 0;

    // Execute
    current_time += output_sequence[i].burst_time;
    output_sequence[i].turnaround_time =
        output_sequence[i].burst_time + output_sequence[i].waiting_time;
    output_sequence[i].completion_time = current_time;

    total_wt += output_sequence[i].waiting_time;
    total_tat += output_sequence[i].turnaround_time;

    printf(" -> [P%d] -> %d", output_sequence[i].pid, current_time);
  }

  printf("\n-------------------\n");

  printf("\nProcess\t AT\t BT\t WT\t TAT\n");
  for (int i = 0; i < n; i++) {
    printf("P%d\t %d\t %d\t %d\t %d\n", output_sequence[i].pid,
           output_sequence[i].arrival_time,
output_sequence[i].burst_time,
           output_sequence[i].waiting_time,
output_sequence[i].turnaround_time);
  }

  printf("\nAverage Waiting Time: %.2f", total_wt / n);
  printf("\nAverage Turnaround Time: %.2f\n", total_tat / n);
}

int main() {
  struct Process p[MAX_PROCESS];
  struct Process sequence[MAX_PROCESS];
  int n, choice;
  int order[MAX_PROCESS];

  printf("Practical 5: FCFS Implementation\n");
  printf("Enter number of processes: ");
  scanf("%d", &n);

  for (int i = 0; i < n; i++) {
    p[i].pid = i + 1;
    printf("\nProcess P%d:\n", i + 1);
    printf("  Enter Arrival Time: ");
```

```
    scanf("%d", &p[i].arrival_time);
    printf("  Enter CPU Burst Time: ");
    scanf("%d", &p[i].burst_time);
  }

  while (1) {
    printf("\n\n--- MENU ---\n");
    printf("1. Standard FCFS (Follow Input/Arrival Sequence)\n");
    printf("2. Custom Sequence\n");
    printf("3. Exit\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    if (choice == 3)
      break;

    if (choice == 1) {
      // Copy directly to sequence
      for (int i = 0; i < n; i++)
        sequence[i] = p[i];
      calculate_times(sequence, n);
    } else if (choice == 2) {
      printf("\nEnter the sequence of Process IDs (e.g., 2 1 3 for P2,
P1, "
             "P3):\n");
      for (int i = 0; i < n; i++) {
        printf("Position %d: P", i + 1);
        int pid_in;
        scanf("%d", &pid_in);

        // Find the process with this PID and add to sequence
        for (int j = 0; j < n; j++) {
          if (p[j].pid == pid_in) {
            sequence[i] = p[j];
            break;
          }
        }
      }
      calculate_times(sequence, n);
    } else {
      printf("Invalid choice.\n");
    }
  }

  return 0;
}
```

Output:

```
❯ gcc pr5.c
~/Documents/CW/OS/pr_4_5
❯ ./a.out
Practical 5: FCFS Implementation
Enter number of processes: 3

Process P1:
  Enter Arrival Time: 1
  Enter CPU Burst Time: 3

Process P2:
  Enter Arrival Time: 2
  Enter CPU Burst Time: 1

Process P3:
  Enter Arrival Time: 3
  Enter CPU Burst Time: 5


--- MENU ---
1. Standard FCFS (Follow Input/Arrival Sequence)
2. Custom Sequence
3. Exit
Enter choice: 1


--- GANTT CHART ---
Time: 0 -> [P1] -> 4 -> [P2] -> 5 -> [P3] -> 10
-------------------

Process  AT      BT      WT      TAT
P1       1       3       0       3
P2       2       1       2       3
P3       3       5       2       7

Average Waiting Time: 1.33
Average Turnaround Time: 4.33


--- MENU ---
1. Standard FCFS (Follow Input/Arrival Sequence)
2. Custom Sequence
3. Exit
Enter choice:
^C
```